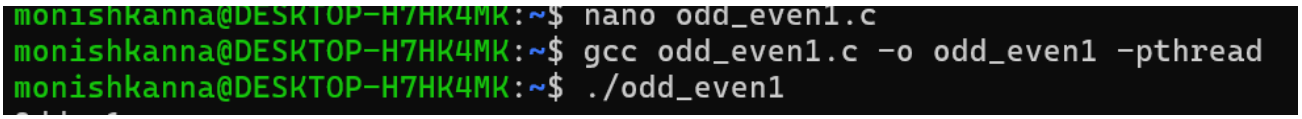
Multithreading - Printing Odd and Even Numbers Using Two Threads

1. Write a C program using POSIX threads where one thread prints odd numbers and another thread prints even numbers up to 100, ensuring the output alternates correctly.

   CODE

```
#include<stdio.h>
#include<pthread.h>
int num=1;
pthread_mutex_t lock;
void* odd(void* arg){
   while(num<=100){
      pthread_mutex_lock(&lock);
      if(num%2!=0){
         printf("Odd: %d\n",num);
         num++;
      }
      pthread_mutex_unlock(&lock);
   }
}
void* even(void* arg){
   while(num<=100){
      pthread_mutex_lock(&lock);
      if(num%2==0){
         printf("Even: %d\n",num);
         num++;
      }
      pthread_mutex_unlock(&lock);
   }
}
int main(){
   pthread_t t1,t2;
   pthread_mutex_init(&lock,NULL);
   pthread_create(&t1,NULL,odd,NULL);
   pthread_create(&t2,NULL,even,NULL);
   pthread_join(t1,NULL);
   pthread_join(t2,NULL);
   pthread_mutex_destroy(&lock);
}
```

   OUTPUT



Implement a program with two threads: one prints odd numbers and the other prints even numbers. Use synchronization primitives (mutexes or condition variables) to avoid race conditions and ensure correct sequence.

CODE

```
#include<stdio.h>
#include<pthread.h>
pthread_mutex_t lock;
int i=1;
void* odd(){
   while(i<=100){
      pthread_mutex_lock(&lock);
      if(i%2==1){
         printf("Odd Thread: %d\n",i++);
      }
      pthread_mutex_unlock(&lock);
   }
}
void* even(){
   while(i<=100){
      pthread_mutex_lock(&lock);
      if(i%2==0){
         printf("Even Thread: %d\n",i++);
```

```
        }
        pthread_mutex_unlock(&lock);
    }
}
int main(){
    pthread_t t1,t2;
    pthread_mutex_init(&lock,NULL);
    pthread_create(&t1,NULL,odd,NULL);
    pthread_create(&t2,NULL,even,NULL);
    pthread_join(t1,NULL);
    pthread_join(t2,NULL);
}
```

OUTPUT

```
monishkanna@DESKTOP-H7HK4MK:~$ nano odd_even2.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc odd_even2.c -o odd_even2 -pthread
monishkanna@DESKTOP-H7HK4MK:~$ ./odd_even2
```

```
Odd Thread: 1
Even Thread: 2
Odd Thread: 3
Even Thread: 4
Odd Thread: 5
Even Thread: 6
Odd Thread: 7
Even Thread: 8
Odd Thread: 9
Even Thread: 10
Odd Thread: 11
Even Thread: 12
Odd Thread: 13
Even Thread: 14
Odd Thread: 15
Even Thread: 16
Odd Thread: 17
Even Thread: 18
Odd Thread: 19
Even Thread: 20
Odd Thread: 21
Even Thread: 22
Odd Thread: 23
Even Thread: 24
Odd Thread: 25
Even Thread: 26
Odd Thread: 27
Even Thread: 28
Odd Thread: 29
Even Thread: 30
Odd Thread: 31
Even Thread: 32
Odd Thread: 33
Even Thread: 34
Odd Thread: 35
Even Thread: 36
Odd Thread: 37
Even Thread: 38
Odd Thread: 39
Even Thread: 40
Odd Thread: 41
Even Thread: 42
Odd Thread: 43
Even Thread: 44
Odd Thread: 45
Even Thread: 46
Odd Thread: 47
Even Thread: 48
Odd Thread: 49
Even Thread: 50
Odd Thread: 51
Even Thread: 52
Odd Thread: 53
Even Thread: 54
Odd Thread: 55
Even Thread: 56
Odd Thread: 57
Even Thread: 58
Odd Thread: 59
Even Thread: 60
Odd Thread: 61
Even Thread: 62
Odd Thread: 63
Even Thread: 64
Odd Thread: 65
Even Thread: 66
Odd Thread: 67
Even Thread: 68
Odd Thread: 69
Even Thread: 70
Odd Thread: 71
Even Thread: 72
Odd Thread: 73
Even Thread: 74
Odd Thread: 75
Even Thread: 76
Odd Thread: 77
Even Thread: 78
Odd Thread: 79
Even Thread: 80
Odd Thread: 81
Even Thread: 82
Odd Thread: 83
Even Thread: 84
Odd Thread: 85
Even Thread: 86
Odd Thread: 87
Even Thread: 88
Odd Thread: 89
Even Thread: 90
Odd Thread: 91
Even Thread: 92
Odd Thread: 93
Even Thread: 94
Odd Thread: 95
Even Thread: 96
Odd Thread: 97
Even Thread: 98
Odd Thread: 99
Even Thread: 100
```

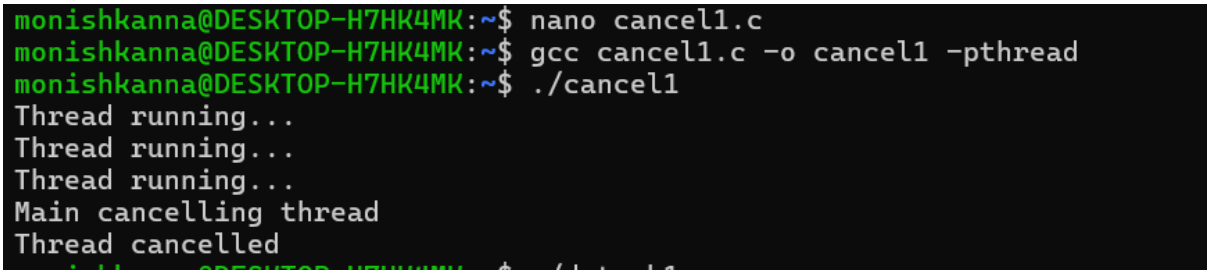
pthread_cancel

1. Write a C program that creates a thread and cancels it from the main thread using pthread_cancel(). Demonstrate how the thread responds to cancellation.

   CODE

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
void* worker(void* arg){
   while(1){
      printf("Thread running...\n");
      sleep(1);
   }
}
int main(){
   pthread_t t;
   pthread_create(&t,NULL,worker,NULL);
   sleep(3);
   printf("Main cancelling thread\n");
   pthread_cancel(t);
   pthread_join(t,NULL);
   printf("Thread cancelled\n");
}
```

   OUTPUT



2. Implement a program where a worker thread performs a long computation. The main thread cancels the worker after a timeout and checks the cancellation status.

   CODE

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
void* worker(void* arg){
   long i=0;
   while(1){
      printf("Processing %ld\n",i++);
      sleep(1); } }
int main(){
   pthread_t t;
   pthread_create(&t,NULL,worker,NULL);
   sleep(4);
   printf("Cancelling worker thread\n");
   pthread_cancel(t);
   void* res;
   pthread_join(t,&res);
   if(res==PTHREAD_CANCELED)
      printf("Worker cancelled successfully\n");
}
```

OUTPUT

```
monishkanna@DESKTOP-H7HK4MK:~$ nano cancel2.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc cancel2.c -o cancel2 -pthread
monishkanna@DESKTOP-H7HK4MK:~$ ./cancel2
Processing 0
Processing 1
Processing 2
Processing 3
Cancelling worker thread
Worker cancelled successfully
```

pthread_detach

1. Write a C program that creates a thread and detaches it using pthread_detach(). Show what happens if the main thread exits before the detached thread finishes.

CODE

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
void* worker(void* arg){
   for(int i=1;i<=5;i++){
      printf("Detached Thread %d\n",i);
      sleep(1);   }  }
int main(){
   pthread_t t;
   pthread_create(&t,NULL,worker,NULL);
   pthread_detach(t);
   printf("Main thread exiting\n");
}
```

OUTPUT

```
monishkanna@DESKTOP-H7HK4MK:~$ nano detach1.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc detach1.c -o detach1 -pthread
monishkanna@DESKTOP-H7HK4MK:~$ ./detach1
Main thread exiting
```

2. Implement a program where multiple threads are created and detached. Demonstrate that their resources are automatically reclaimed upon termination.

CODE

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
void* worker(void* arg){
   int id=(int)(long)arg;
   printf("Thread %d running\n",id);
   sleep(2);
   printf("Thread %d finished\n",id);
}
int main(){
   pthread_t t[3];
   for(int i=0;i<3;i++){
      pthread_create(&t[i],NULL,worker,(void*)(long)i);
      pthread_detach(t[i]);
   }
   sleep(5);
   printf("Main finished\n");}
```

OUTPUT

```
monishkanna@DESKTOP-H7HK4MK:~$ nano detach2.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc detach2.c -o detach2 -pthread
monishkanna@DESKTOP-H7HK4MK:~$ ./detach2
Thread 0 running
Thread 1 running
Thread 2 running
Thread 1 finished
Thread 2 finished
Thread 0 finished
Main finished
```

pthread_join

1. Write a C program that creates several threads and uses pthread_join() to wait for each to finish, collecting their return values.

   CODE

```
#include<stdio.h>
#include<pthread.h>
void* worker(void* arg){
   int num=(int)(long)arg;
   int result=num*num;
   return (void*)(long)result; }
int main(){
   pthread_t t[3];
   void* res;
   for(int i=1;i<=3;i++)
      pthread_create(&t[i-1],NULL,worker,(void*)(long)i);
   for(int i=0;i<3;i++){
      pthread_join(t[i],&res);
      printf("Thread result: %ld\n",(long)res);
   }
}
```

   OUTPUT

```
monishkanna@DESKTOP-H7HK4MK:~$ nano join1.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc join1.c -o join1 -pthread
monishkanna@DESKTOP-H7HK4MK:~$ ./join1
Thread result: 1
Thread result: 4
Thread result: 9
```

2. Implement a program where the main thread waits for a worker thread to complete using pthread_join(), and prints the result returned by the worker.

   CODE

```
#include<stdio.h>
#include<pthread.h>
void* worker(void* arg){
   int n=5;
   return (void*)(long)(n*2);}
int main(){
   pthread_t t;
   void* result;
   pthread_create(&t,NULL,worker,NULL);
   pthread_join(t,&result);
   printf("Worker returned %ld\n",(long)result);}
```

OUTPUT

```
monishkanna@DESKTOP-H7HK4MK:~$ nano join2.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc join2.c -o join2 -pthread
monishkanna@DESKTOP-H7HK4MK:~$ ./join2
Worker returned 10
```

pthread_exit

1. Write a C program where a worker thread uses pthread_exit() to return a value to the main thread. The main thread should retrieve and print this value using pthread_join().

CODE

```
#include<stdio.h>
#include<pthread.h>
void* worker(void* arg){
   int value=25;
   pthread_exit((void*)(long)value); }
int main(){
   pthread_t t;
   void* res;
   pthread_create(&t,NULL,worker,NULL);
   pthread_join(t,&res);
   printf("Thread returned %ld\n",(long)res); }
```

OUTPUT

```
monishkanna@DESKTOP-H7HK4MK:~$ nano exit1.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc exit1.c -o exit1 -pthread
monishkanna@DESKTOP-H7HK4MK:~$ ./exit1
Thread returned 25
```

2. Implement a program that creates multiple threads, each calling pthread_exit() with a different exit status. The main thread should collect and display the exit status of each thread.

CODE

```
#include<stdio.h>
#include<pthread.h>
void* worker(void* arg){
   int id=(int)(long)arg;
   pthread_exit((void*)(long)(id*10)); }
int main(){
   pthread_t t[3];
   void* res;
   for(int i=1;i<=3;i++)
      pthread_create(&t[i-1],NULL,worker,(void*)(long)i);
   for(int i=0;i<3;i++){
      pthread_join(t[i],&res);
      printf("Thread %d exit status %ld\n",i+1,(long)res);    } }
```

OUTPUT

```
monishkanna@DESKTOP-H7HK4MK:~$ nano exit2.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc exit2.c -o exit2 -pthread
monishkanna@DESKTOP-H7HK4MK:~$ ./exit2
Thread 1 exit status 10
Thread 2 exit status 20
Thread 3 exit status 30
```